

Le handicap

1 Définition du handicap

« Une personne handicapée est une personne qui présente des **incapacités durables** combinées à des **barrières** qui empêchent sa **pleine et effective participation à la société sur la base de l'égalité avec les autres** » (définition de la Convention relative aux droits des personnes handicapées).
L'accessibilité est un **droit fondamental**.

3 Types de handicap

- Handicap physique (handicap moteur)
- Handicap sensoriel
 - handicap visuel (non-voyance, malvoyance, daltonisme etc.)
 - handicap auditif
- Handicap mental
 - handicap intellectuel
 - handicap cognitif (troubles Dys par ex.)
 - handicap psychique (bipolarité, schizophrénie, troubles anxieux etc.)
- Cancer, douleurs chroniques

2 Caractéristiques

Le handicap est **varié, multiple et cumulable**.
Chaque personne est **différente**.

4 La situation de handicap

Une personne en **situation de handicap** rencontre des obstacles qui empêchent ses activités (se déplacer, travailler etc.). La situation de handicap est **variable et évolue** dans le temps. **Un environnement inadapté entraîne une situation de handicap**.

5 Textes de loi

- Convention relative aux droits des personnes handicapées (ONU)
- Loi du 11 février 2005 (et particulièrement son article 47) pour l'égalité des droits et des chances, la participation et la citoyenneté des personnes handicapées

L'accessibilité numérique

1 Objectif de l'accessibilité numérique

Permettre à **tout individu** de profiter du Web et des ses services **quel que soit** leur matériel ou logiciel, infrastructure réseau, langue maternelle, culture, localisation géographique et aptitudes physiques ou mentales, ou leur âge (les personnes âgées ont aussi le droit d'utiliser le Web).

3 Des besoins spécifiques

Pouvoir personnaliser l'affichage (couleurs, taille du texte, police d'affichage), naviguer seulement au clavier ou seulement à la souris, naviguer à son rythme... Les besoins des utilisateurs sont **multiples et variés**.

96,3%

96% des sites Internet ne sont pas accessibles

2 L'inclusion grâce au numérique

Faire ses courses, payer ses impôts, se divertir, exprimer ses opinions...

Un monde numérique accessible est un **vecteur d'inclusion et d'autonomie**.

4 Des technologies d'assistance (TA)

Permettre d'accéder à l'information par un autre moyen. Elles sont multiples : plage braille, lecteur d'écran, clavier visuel, trackball, contacteur au souffle, headstick etc.

Elles ne **rendent pas** un site accessible mais sont un **complément** à un site accessible.

Une technologie d'assistance **ne peut fonctionner correctement** avec un site non accessible.

Parfois, il n'existe pas de technologie d'assistance pour aider une personne handicapée à accéder à l'information.



Une plage braille

5 Normes et référentiels

- **UUAG** (User Agent Accessibility Guidelines) : directives pour rendre des logiciels permettant d'afficher du contenu Web accessibles (navigateurs, extensions pour navigateur, lecteurs audio/vidéo etc.)
- **ATAG** (Authoring Tool Accessibility Guidelines) : directives pour rendre les outils de contribution (outils de WYSIWYG, CMS etc.) accessibles
- **WCAG** (Web Content Accessibility Guidelines) : standards pour créer du contenu Web accessible (actuellement en version 2.1)
- **ARIA** (Accessible Rich Internet Applications) : permet de créer du contenu dynamique et des composants développés en HTML/CSS/JS accessibles

6 Niveaux d'accessibilité

Les WCAG définit trois niveaux d'accessibilité :

- A : accès à l'information et utilisabilité minimum (80% des critères)
- AA : critères un peu plus exigeants, **le niveau AA est le niveau d'accessibilité à atteindre** (le niveau A étant contenu dans le niveau AA)
- AAA : niveau très exigeant, difficile à mettre en application sur des sites entiers

7 Le Référentiel général d'amélioration de l'accessibilité

Le Référentiel général d'amélioration de l'accessibilité (RGAA) est une **application franco-française** plus claire des WCAG. 106 critères A/AA sur 13 thématiques associés à des tests pour vérifier si un site est accessible.

8 Définition d'un critère du RGAA

Un critère est constitué :

- d'une description (par exemple « Chaque image porteuse d'information a-t-elle une alternative textuelle ? »)
- des tests pour vérifier si le critère est respecté ou non
- les critères du WCAG associés
- éventuellement des cas particuliers

Hors cas particuliers, un test d'un critère qui n'est pas respecté invalide le critère.

10 Niveaux de conformité

- « Accessibilité : totalement conforme » si 100% des critères sont respectés
- « Accessibilité : partiellement conforme » si au moins 50% des critères sont validés
- « Accessibilité : non conforme » si aucun d'audit d'accessibilité n'a été réalisé ou que moins de 50% des critères sont validés

9 Présence et pertinence

La **présence** : est-ce que c'est bien **mis en œuvre** ?

La **pertinence** : est-ce que c'est **pertinent** ?

Par exemple, le critère 1.1 vérifie la **présence** d'une alternative textuelle sur les images porteuses d'information.

Le critère 1.3 vérifie si l'alternative textuelle est **pertinente**.

11 Les obligations du RGAA

- Afficher son niveau de conformité au RGAA sur la page d'accueil
- À permettre d'accéder depuis n'importe quelle page du site à la déclaration d'accessibilité
- À permettre d'accéder au schéma pluriannuel de mise en conformité si le site n'est pas conforme

Une amende pouvant aller jusqu'à 20 000 € par site et par an est prévue si une des ces obligations n'est pas respectée.

12 La déclaration d'accessibilité

La déclaration d'accessibilité doit contenir :

- le niveau de conformité
- la liste des contenus non accessibles
- des moyens de contact
 - à l'administration du site pour pouvoir signaler des défauts d'accessibilité et accéder à l'information non accessible
 - au Défenseur des droits si la réponse de l'administration du site n'est pas satisfaisante ou en cas d'absence de réponse

13 L'accessibilité numérique dès la conception d'un projet

L'accessibilité numérique doit être prise en compte durant les premières phases d'un projet.

Prise en compte le plus tôt possible, les coûts liés à l'accessibilité numérique ne sont qu'en moyenne de 20% en plus (ils sont de 40% supplémentaires si elle prise en compte tardivement dans la conception du projet).

L'accessibilité numérique a des impacts sur toutes les parties prenantes : les chefs de projet doivent être conscients qu'elle n'est pas une option, les designers doivent la prendre en compte quand ils réalisent leurs maquettes et les développeurs doivent développer accessible.

Le client doit aussi être conscient de l'accessibilité numérique, au-delà des enjeux juridiques.

L'accessibilité numérique

4 grands principes

Perceptible

Je perçois l'information de multiples façons, quelle que soit la situation de handicap : alternatives textuelles des images, information pas seulement donnée par la couleur, contrastes suffisants etc.

Utilisable

Je peux interagir avec la plateforme/site Web de la façon qui correspond à mes besoins : seulement au clavier, seulement à la souris, je peux mettre en pause un carrousel d'images etc.

Compréhensible

L'information est facile à comprendre, je peux interagir avec la plateforme/site Web de façon intuitive : mes zones de navigation sont toujours à la même place, mes messages d'erreurs sont clairs etc.

Robuste

Ma plateforme/mon site Web peut être utilisé avec des technologies d'assistance et est testé pour assurer qu'il soit toujours accessible avec le temps et les évolutions des navigateurs et des technologies d'assistance.

Que retenir ?

L'accessibilité numérique est **fondamentale**, tous les individus doivent pouvoir jouir des services offerts par le Web.

Tout le monde est **différent**. Chacun a des **besoins spécifiques**.

Les technologies d'assistance sont un **complément** à un site accessible.

L'accessibilité numérique est faite de normes et de référentiels. En France, c'est le **RGAA** (Référentiel général d'amélioration de l'accessibilité) qui prévaut.

Le RGAA est composé de critères à respecter pour qu'un site soit considéré comme accessible. Il oblige les propriétaires des sites Web à afficher le niveau de conformité et à publier une déclaration d'accessibilité.

L'accessibilité doit être prise en compte dès la conception du projet. Toutes les parties prenantes d'un projet doivent être conscientes, au-delà des conséquences juridiques, que l'accessibilité n'est pas une option.

Elle nécessite des professionnels formés et représente un coût certain mais plus élevé si elle est prise en compte tardivement.

Le nom et la description accessible

Le **nom accessible** est une **description courte associée à un élément utilisée par les technologies d'assistance pour décrire l'élément**.

Il peut être **l'intitulé visible de l'élément** (d'un bouton ou d'un lien par exemple) ou un **intitulé invisible** (l'alternative textuelle d'une image par exemple ou la valeur de l'attribut *aria-label* ou *aria-labelledby*).

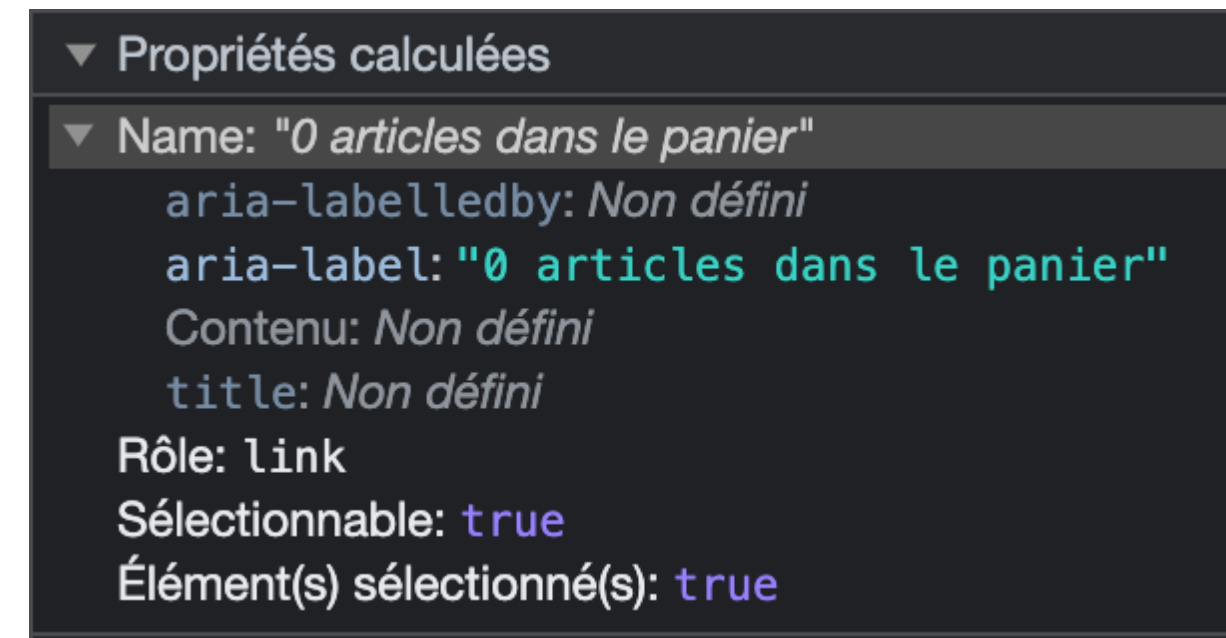
Un lecteur d'écran dira « bouton "Valider" » si le bouton a pour intitulé « Valider ».

Le nom accessible contient aussi la **description accessible** (attribut *aria-describedby*) qui sert de **complément** au nom accessible : un bouton « Ajouter le produit » ayant une description accessible « Il reste 3 exemplaires en stock » sera vocalisé « bouton "Valider Il reste 3 exemplaires en stock" ».

Le nom accessible est **calculé différemment en fonction des éléments HTML** : [Accessible Name Computations By HTML Element](#).

Le navigateur va suivre cet algorithme pour calculer le nom accessible.

Dans une majorité des cas, les attributs ARIA *aria-label* et *aria-labelledby* sont utilisés en priorité, puis le libellé de l'élément, pour finir par d'autres attributs comme les attributs *alt*, *title*, *placeholder* en fonction du type d'élément HTML.



Nom accessible sur Chrome

Dans l'inspecteur d'éléments de Chrome (similairement dans Firefox), le nom accessible est visible dans le sous-onglet Accessibilité, dans « Name ».

On peut voir dans cet exemple que le lien menant au panier sur le site d'Amazon a pour nom accessible « 0 articles dans le panier ».

Les couleurs et contrastes

1 Le contraste

Pour que le contenu puisse être visible et lisible pour le plus grand monde, il faut que le rapport de contraste entre l'arrière-plan et le contenu soit suffisant.

Attention, c'est technique.

3 Le contraste du contenu non textuel

Pour les images, icônes, composants d'interface etc. :

- rapport de **3,1:1**
- **mécanisme** qui permet d'obtenir un rapport de contraste suffisant

Attention aux cas particuliers.

Pour plus d'informations : [critère 3.3 du RGAA](#).

2 Le contraste du contenu textuel

Pour le texte, le texte dans des images, sous-titres dans une vidéo etc. :

Taille < **24px** ou **18,5px en gras** : rapport de **4,5:1**

Taille >= **24px** ou **18,5px en gras** : rapport de **3,1:1**

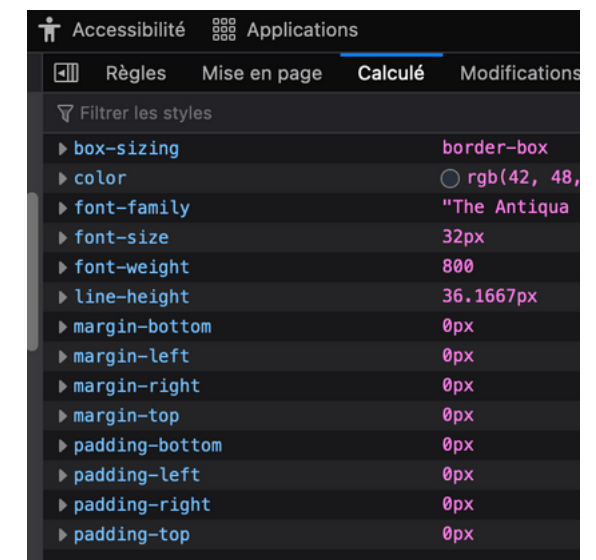
La taille dont on parle ici est la **taille calculée** et non de la taille spécifiée dans la feuille de style CSS.

Attention aux cas particuliers.

Pour plus d'informations : [critère 3.2 du RGAA](#).

4 Calculer le rapport de contraste

- Logiciel [Colour Contrast Analyzer](#)
- Site [Contrast Checker](#)



Taille calculée sur Firefox

5 L'information par la couleur

On ne peut pas donner de l'information que par la couleur.

Les personnes aveugles et déficientes visuelles ne peuvent percevoir et comprendre l'information véhiculée par la couleur : il faut donc **doubler l'information** avec une alternative.

Les cas les plus courants d'information donnée par le couleur sont la page active, un élément sélectionné (typiquement un onglet), une bordure rouge pour indiquer un champ en erreur par exemple.

6 Alternatives à l'information par la couleur

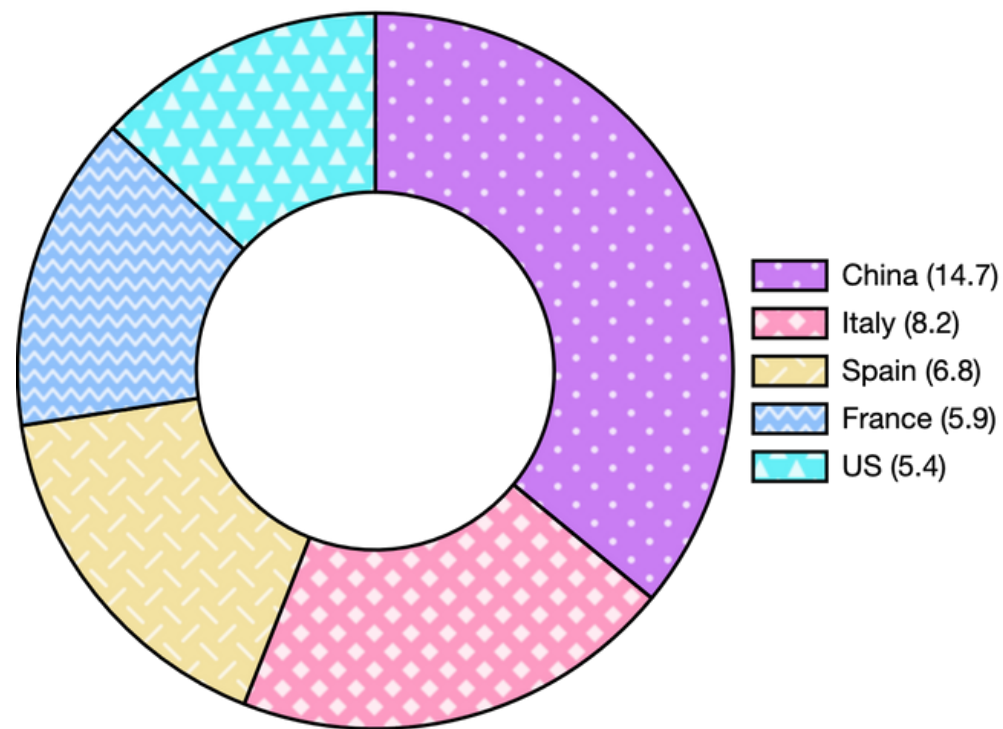
Attributs *title* ou ARIA *aria-current*, *aria-label*, *aria-describedby* ou utilisation de texte visible que par les technologies d'assistance :

```
<a href="#" title="Accueil - page active">Accueil</a>  
<a href="#" aria-current="page">À propos de nous</a>
```

JUILLET 2023						
LUN.	MAR.	MER.	JEU.	VEN.	SAM.	DIM.
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31						

Exemple dans lequel le taux d'occupation en fonction des jours d'un parc d'attractions est donné par la couleur.

On peut utiliser du texte visible que par les technologies d'assistance ou ajouter encore plus d'information en utilisant des formes ou des motifs pour permettre la distinction entre les différentes couleurs.



Exemple de graphique dans lequel la couleur est doublée par des motifs permettant ainsi de mieux distinguer les différents pourcentages.

5

Cacher un élément mais le rendre visible aux technologies d'assistance

Un élément rendu non visible avec du CSS (avec *display: none*) 'est également par visible par les technologies d'assistance. On peut utiliser une classe spéciale généralement nommée *.sr-only* (pour "screen reader only") ou *.visually-hidden*. Ci-dessous un exemple de classe *.sr-only* améliorée, de Gaël Poupard ([source sur GitHub](#)) :

```
.sr-only {
  border: 0 !important;
  clip: rect(1px, 1px, 1px, 1px) !important;
  -webkit-clip-path: inset(50%) !important;
  clip-path: inset(50%) !important;
  height: 1px !important;
  margin: -1px !important;
  overflow: hidden !important;
  padding: 0 !important;
  position: absolute !important;
  width: 1px !important;
  white-space: nowrap !important;
}
```

Que retenir ?

Les éléments d'une page Web doivent être suffisamment être contrastés pour être visibles et lisibles.

Le rapport de contraste entre l'arrière-plan et du **contenu non textuel** (images, icônes, composants d'interface etc.) doit être de **3,1:1**.

Le rapport de contraste entre l'arrière-plan et du **contenu textuel** doit être compris entre **3,1:1 et 4,5:1 en fonction de la taille et de la graisse**.

De l'information ne peut être donnée que par de la couleur.

Toujours **doubler l'information par la couleur d'une alternative** ou éventuellement d'une forme ou d'un motif pour permettre de comprendre l'information.

Cacher un élément avec du CSS (avec une règle *display: none* par exemple) empêche les technologies d'assistance de pouvoir accéder à cet élément, ce qui peut faire **perdre de l'information**.

La très grande majorité des librairies CSS proposent toutes des classes CSS dédiées à rendre non visible du contenu mais accessible aux technologies d'assistance.

Les images

1 Images et icônes de décoration ou porteuses d'information

Une image (ou icône) de décoration est une image qui ne fait que **décorer** le texte environnant, elle n'apporte aucune information.

Elles doivent être **ignorées par les technologies d'assistance**.

Une image (ou icône) porteuse d'information **contient ou porte des informations**, elle doit donc avoir une **alternative** pour permettre aux utilisateurs déficients visuels aient accès à ces informations.

Pour plus d'informations : [critère 1 du RGAA](#).

2 Définir une alternative

```
  
  
  
  
  
<span id="alternative">Mon alternative textuelle</span>
```

3 Cacher une image de décoration aux technologies d'assistance

```
  
  
  
  
<svg aria-hidden="true">...</svg>
```

Les images

5 Les SVG porteurs d'information

Les SVG porteurs d'information en ligne (qui ne sont pas dans une balise `<img>`) :

```
<svg role="img" aria-label="Alternative textuelle">
  ...
</svg>
<svg role="img" aria-labelledby="alternative">
  <title id="alternative">Alternative textuelle</title>
  ...
</svg>
```

Pour les SVG utilisés avec une balise `<img>` :

```


```

7 CAPTCHA

Les CAPTCHA doivent disposer d'une alternative sonore ou doivent permettre d'obtenir le code d'une autre façon (par SMS par exemple).

Critères 1.4 et 1.5 du RGAA.

6 Images complexes

Une image complexe (une infographie par exemple) doit avoir une alternative textuelle décrivant de façon détaillée le contenu de l'image : à côté de l'image (en-dessous généralement) ou accessible en cliquant sur un lien ou un bouton.

Critères 1.6 et 1.7 du RGAA.

8 Texte dans des images

Le texte dans des images qui donne de l'information doit être évité et remplacé par du texte HTML/CSS classique de façon à ce que les personnes qui nécessitent d'adapter la manière dont le texte est affiché puissent le faire (modifier la taille, la couleur, l'espacement des caractères etc.).

Si ce n'est pas possible, un mécanisme de remplacement accessible doit être proposé.

Critère 1.8 du RGAA.

5 Images légendées

```
<figure role="group" aria-label="Légende de l'image">
  
  <figcaption>
    Légende de l'image
  </figcaption>
</figure>
```

Que retenir ?

Deux types d'images : porteuses d'information et de décoration.

Les images porteuses d'information **doivent avoir une alternative textuelle** pour que les personnes déficientes visuelles aient accès à ces informations.

Les images de décoration **doivent être ignorées** par les technologies d'assistance.

L'alternative textuelle constitue le **nom accessible** de l'image.

Les CAPTCHA doivent avoir une alternative sonore ou permettre de récupérer le code affiché d'une autre façon pour que les utilisateurs ne soient pas bloqués dans la complétion d'un formulaire.

Les images complexes, comme une infographie, doivent avoir une alternative qui reprend leur contenu, accessible facilement.

Parce que certaines personnes ont besoin d'adapter la façon dont le texte est affiché, le texte dans des images doit être évité.

Les composants d'interface

1 Les composants d'interface

Un composant d'interface est un lien, un bouton, un champ de formulaire mais également tout composant plus avancé comme une fenêtre de dialogue, un menu ou encore un accordéon.

Tout élément avec lequel l'utilisateur peut interagir est un composant d'interface.

Définition du RGAA

3 Les boutons

Les boutons (balise `<button>`) servent **uniquement à exécuter une action** (par exemple, exécuter une fonction JS qui ajoute un produit dans un panier).

Comme pour les liens, ils doivent avoir **un intitulé** (à défaut d'intitulé visible, une alternative pertinente).

La fonction JS devra être évaluée dans la thématique 7 du RGAA dédiée aux scripts JS pour s'assurer qu'elle est bien accessible.

2 Les liens

Les liens (balise `<a>`) doivent avoir **un intitulé visible** et celui-ci doit être **pertinent** pour comprendre vers où il mène.

Un lien ne peut être utilisé que pour naviguer d'une page à une autre. Un lien doit être obligatoirement souligné.

Dans le cas des liens contenant des images (un lien image), c'est l'alternative textuelle des images qui font office d'intitulé. D'autres types de liens et des cas particuliers existent : critères 6 du RGAA.

4 Ça, c'est un bon composant d'interface

Un bon composant d'interface :

- doit être **accessible à la navigation au clavier**
- doit être **activables au clavier** (un item d'accordéon peut être ouvert en appuyant sur Entrée ou Espace par exemple)
- ce qui **apparaît au survol doit aussi apparaître au focus**
- ce qui est **ouvert doit pouvoir être fermé au clavier** (un item d'accordéon par exemple)
- doit être **bien contrasté** (en prenant en compte ses états)

Hiérarchisation du contenu

1 Hiérarchisation du contenu

Un **contenu bien hiérarchisé** permet aux personnes utilisant des technologies d'assistance de pouvoir **naviguer plus facilement** sur la page et **accéder rapidement au contenu qui les intéresse**.

On utilise pour cela les balises `<h1>` (titre de niveau 1, plus important) à `<h6>` (titre de niveau 6, le moins important).

Pour le RGAA, le **critère 9.1 est validé** même si :

- il n'y a pas de titres (parfois il n'y en pas besoin)
- il n'y a pas de titre de niveau 1
- il y a plusieurs titres de niveau 1
- il y a des sauts de niveau (un `<h1>` puis un `<h3>` par exemple)

```
<h1>Les chiens</h1>
  <h2>Les races de chiens</h2>
  <h2>Comment s'occuper de son chien ?</h2>
    <h3>Le toilettage</h3>
```

Organisation du contenu d'un article

```
<h1>Tout sur les chiens</h1>
  <h2>Menu de navigation</h2>
  <h2>Les chiens</h2>
    <h3>Les races de chiens</h3>
    <h3>Comment s'occuper d'un chien ?</h3>
  <h2>Pied de page</h2>
```

Organisation d'une page avec le `<h1>` en premier

```
<h2>Menu de navigation</h2>
<h1>Tout sur les chiens</h1>
  <h2>Les chiens</h2>
    <h3>Les races de chiens</h3>
    <h3>Comment s'occuper d'un chien ?</h3>
  <h2>Pied de page</h2>
```

Organisation d'une page avec le `<h1>` après le menu

1 Régions et autres balises sémantiques

Une région est une partie d'une page Web. Elles permettent aux technologies d'assistance mais aussi à certaines extensions navigateur de pouvoir aider l'internaute à accéder aux parties de la page qui l'intéresse :

- balise `<header>` : en-tête de la page où l'on place le logo, le menu de navigation principale, un système de recherche
- balise `<nav>` : permet de définir des menus de navigation
- balise `<main>` : contient le contenu principal de la page (**obligatoire, une seule balise `<main>` par page**)
- balise `<footer>` : pied de page contenant généralement les mentions légales et la déclaration d'accessibilité

La spécification HTML 5 a ajouté d'autres balises sémantiques :

- balise `<aside>` : contient du contenu complémentaire (typiquement une liste d'articles connexes)
- balise `<article>` : représente généralement du contenu indépendant comme un article de blog, un commentaire etc.
- balise `<section>` : divise en sections le contenu de la page

```
<body>
  <header role="banner">
    <h1>Tout sur les chiens</h1>
  </header>
  <nav aria-labelledby="menu-navigation">
    <h2 id="menu-navigation">Menu de navigation</h2>
    <ul>
      <li><a href="#">Accueil</a></li>
      <li><a href="#">Contact</a></li>
    </ul>
  </nav>
  <main>
    <article>
      <h2>Comment s'occuper d'un chien ?</h2>
      <section>
        <h3>Le toilettage</h3>
        <p>...</p>
      </section>
      <aside aria-labelledby="articles-connexes">
        <h3 id="articles-connexes">Articles connexes</h3>
        <ul>
          <li><a href="#">Le berger allemand</a></li>
          <li><a href="#">Le labrador</a></li>
        </ul>
      </aside>
    </article>
  </main>
  <footer role="contentinfo">
    <ul>
      <li><a href="#">Déclaration d'accessibilité</a></li>
      <li><a href="#">Mentions légales</a></li>
    </ul>
  </footer>
</body>
```

Exemple de page correctement structurée avec des régions

2 Règles relatives aux régions

- L'en-tête principal de la page doit avoir le rôle ARIA *banner*, il ne peut avoir qu'un seul en-tête ayant un rôle ARIA *banner* dans une page
- Si la page contient un ou plusieurs systèmes de recherches, ils doivent avoir un rôle ARIA *search* et un attribut *aria-label* ou *aria-labelledby* surtout s'il y a plusieurs systèmes de recherche dans la page
- Il ne peut y avoir qu'une seule balise `<main>` dans une page
- Les menus de navigations doivent contenir des listes et un attribut *aria-label* ou *aria-labelledby* surtout s'il y a plusieurs menus de navigation dans la page
- Un menu situé dans le pied de page n'a pas besoin d'être placé dans une balise `<nav>`
- Le pied de page doit avoir un rôle ARIA *contentinfo*

3 Type, langue et titre du document

Une page HTML **doit toujours commencer par un type de document** (doctype).

En HTML 5, le type de document est `<!doctype html>`.

Spécifier l'attribut *lang* de la balise `<html>` avec la bonne langue (*lang="fr-FR"* pour une page en français).

L'attribut *lang* peut être ajouté à n'importe quelle balise pour indiquer aux technologies d'assistance que le mot n'est pas écrit dans la langue de la page (critère 8.8 du RGAA).

Une page HTML doit également avoir une balise `<title>` contenant le titre de la page.

```
<!doctype html>
<html lang="fr-FR">
<head>
  <meta charset="UTF-8">
  <title>Titre de la page</title>
</head>
<body>
  ...
</body>
</html>
```


Structure et sémantique du contenu

4 Paragraphes

Petit rappel : les divisions (balise `<div>`) sont des **conteneurs de données**, elles **ne doivent pas être utilisées pour contenir du texte**.

Pour créer un paragraphe, il faut utiliser la balise `<p>`.

```
<div>
  
  <p>...</p>
</div>
```

6 Les citations

Pour des citations longues, utiliser la balise `<blockquote>`.

Pour des citations courtes, incluses dans un autre contenu, utiliser la balise `<q>`.

La balise `<cite>` ou l'attribut `cite` sert à indiquer la source de la citation.

Critère 9.4 du RGAA.

5 Les listes

Pour rappel, deux types de listes :

- liste ordonnée (balise `<ol>`)
- liste non ordonnée, pour du contenu séquentiel (balise `<ul>`)

Un item de liste est créé grâce à la balise `<li>`.

Critère 9.3 du RGAA

```
<ul>
  <li>Labrador</li>
  <li>Golden retriever</li>
</ul>

<ol>
  <li>Labrador</li>
  <li>Golden retriever</li>
</ol>
```

```
<blockquote>
  <p>Be you, be proud of you because you can be do what we
  want to do</p>
  <footer>—François Hollande, <cite>Conférence mondiale sur
  le climat à Paris (2015)</cite></footer>
</blockquote>
```

4 Les cadres (iframes)

Les cadres ou iframes servent à intégrer un document HTML externe dans une page.

Les iframes doivent avoir un attribut *title* décrivant le but du cadre, celui-ci doit être pertinent.

Critères 2 du RGAA

5 Liens d'accès rapide

Chaque page doit être avoir un lien d'accès rapide qui permet à l'utilisateur d'accéder directement au contenu principal de la page.

Ce lien doit être **visible à la prise de focus** (mais être caché de façon accessible) et lorsqu'il est cliqué, **le focus doit être déplacé sur le contenu**.

[Accéder au contenu principal](#)

Bienvenue ! [Connectez-vous](#) ou [inscrivez-vous](#)

eBay

 Explorer par   Reche

Lien d'accès rapide du site eBay

Que retenir ?

Les pages HTML doivent correctement être structurées avec des régions.

Une page bien structurée et bien hiérarchisée a des bénéfices pour tout le monde : les technologies d'assistance et certaines extensions navigateur peuvent permettre aux internautes d'accéder seulement au contenu qui les intéresse, le contenu peut être cherché plus facilement, les navigateurs dotés d'un « mode lecture » retirant tout le superflu pour ne laisser que le contenu ne fonctionne seulement si la page est bien structurée et hiérarchisée.

Une page HTML doit commencer par un type de document.

La langue et le titre de la page doivent être définis.

Utiliser au maximum des balises sémantiques, qui ont du sens pour construire une page HTML, au lieu de balises génériques.

Chaque page doit avoir un lien d'accès rapide pour permettre aux utilisateurs qui naviguent au clavier et au lecteur d'écran de pouvoir accéder rapidement au contenu principal de la page.

1 Quelques règles relatives aux formulaires

- Tous les champs des formulaires **doivent être placés dans une balise `<form>`**, les technologies d'assistance ont un mode dédié aux formulaires
- **Les champs obligatoires doivent avoir un attribut `required`** (le caractère obligatoire du champ peut être ajouté dans le libellé du champ s'il n'y pas de mention relative aux champs obligatoires)
- On ne place pas une mention « * Champs obligatoires » ou « Tous les champs obligatoires » après le formulaire, les instructions nécessaires (une liste de pièces justificatives par exemple) au remplissage du formulaire **doivent être placées avant** celui-ci (ce qui n'empêche pas les champs d'avoir une description indiquant un format particulier par exemple)
- On ne met pas de balise `<input>` dans une balise `<label>`, certaines technologies d'assistance ne le supportent pas

2 Les libellés des contrôles de formulaire

Un champ doit avoir un **libellé visible** (cela garantit que l'on puisse savoir à tout moment la fonction du champ) et **pertinent**. Le *placeholder* d'un champ **ne constitue pas** un libellé puisqu'il disparaît à la saisie.

À défaut, l'attribut *title* ou les attributs ARIA *aria-label* ou *aria-labelledby* peuvent être utilisés.

Un libellé de champ **doit être accolé au champ de formulaire** (de façon à ce qu'un utilisateur qui a besoin de zoomer dans la page puisse voir le libellé).

Critères 11.1 et 11.2 du RGAA

```
<label for="prenom">Prénom</label>
<input type="text" name="prenom" id="prenom">

<p id="description">Prénom</p>
<input type="text" aria-labelledby="description" />

<p>Prénom</p>
<input type="text" aria-label="Inscrivez votre prénom" />

<input title="Votre prénom" type="text" id="prenom" />
```

3 Les regroupements

La balise `<fieldset>` permet de regrouper des champs par thématique : par exemple, grouper tous les champs relatifs à l'adresse de facturation.

Elle contient une balise `<legend>` qui permet d'identifier le regroupement.

```
<fieldset>
  <legend>Adresse de facturation</legend>
  <div>
    <label for="rue">Rue</label>
    <input type="text" name="rue" id="rue" />
  </div>
  <div>
    <label for="code_postal">Code postal</label>
    <input type="text" name="code_postal" id="code_postal" />
  </div>
</fieldset>
```

Inscription universitaire

Parent 1

Prénom

Nom

Parent 2

Prénom

Nom

Annuler

S'inscrire

Lorsque je navigue dans ce formulaire, comment puis-je savoir pour quel parent je renseigne les informations ?

Les formulaires

4 Les listes de sélection

S'assurer que les groupements d'options d'une liste de sélection portent bien un libellé.

```
<label for="pays">Pays</label>
<select name="pays" id="pays">
  <optgroup label="Europe">
    <option value="france">France</option>
    <option value="allemagne">Allemagne</option>
  </optgroup>
  <optgroup label="Amérique du Nord">
    <option value="etats-unis">États-Unis</option>
    <option value="canada">Canada</option>
  </optgroup>
</select>
```

Pays

Europe	
✓	France
	Allemagne
Amérique du Nord	
	États-Unis
	Canada

Groupements d'options

5 Boutons

S'assurer que les boutons aient un **nom accessible pertinent**.

Un bouton `<button>` et un `<input>` de type `submit` servent à soumettre le formulaire. Un `<input>` sans type sera de type `text` par défaut.

Un `<button>` sans type sera de type `submit` par défaut.

```
<button type="submit">Valider</button>
<input type="button" aria-label="Valider" />

<input type="image" alt="Valider" />
<input type="submit" value="Valider" />
<input type="submit" title="Valider" />

<p id="valider">Valider</p>
<input type="button" aria-labelledby="valider" />
```


6 Aider à la saisie

Demander à l'utilisateur de renseigner une date c'est bien mais comment doit-elle être renseignée ? Au format JJ/MM/AAAA ?

Au format MM/JJ/AAAA ?

Il faut **accompagner l'utilisateur** pour qu'il remplisse votre formulaire correctement.

Vous pouvez indiquer le format de la donnée attendue dans le libellé ou avec une description accessible.

La description du format doit être placée avant le champ.

Le format de la donnée attendue ne doit pas être placé dans le placeholder, il disparaît à la saisie.

```
<label for="date_naissance">Date de naissance (au format  
JJ/MM/AAAA)</label>  
<input type="date" name="date_naissance" id="date_naissance">  
  
<label for="date_naissance">Date de naissance</label>  
<p id="format_date_naissance">Format JJ/MM/AAAA</p>  
<input type="date" name="date_naissance" id="date_naissance"  
aria-describedby="format_date_naissance">
```

7 Messages d'erreur

Utiliser l'attribut ARIA *aria-invalid* pour indiquer que le champ est invalide.

Comme pour l'aide à la saisie, les messages d'erreur sont liés au champ avec un attribut ARIA *aria-describedby*.

Si la validation se fait dynamiquement (avec une fonction JS), il faut demander au lecteur d'écran de vocaliser le message d'erreur immédiatement en utilisant l'attribut ARIA *aria-live* avec la valeur *assertive* pour que l'utilisateur corrige son erreur.

Le message d'erreur doit être explicite : rappeler le format de la donnée attendue.

```
<label for="date_naissance">Date de naissance (format JJ/MM/AAAA)</label>  
<input type="date" name="date_naissance" id="date_naissance" aria-  
describedby="date_naissance_incorrecte" aria-invalid="true">  
  
<p id="date_naissance_incorrecte">La date de naissance est invalide, le  
format doit être JJ/MM/AAAA</p>
```

8 Le pré-remplissage par le navigateur

Le navigateur peut pré-remplir les champs automatiquement avec les informations qu'il a en mémoire.

Il se base pour cela sur l'attribut *autocomplete* de la balise `<input>`.

Cet attribut peut prendre plusieurs valeurs listées dans la spécification HTML.

Si la valeur est *on*, le navigateur va remplir essayer de remplir le champ tout seul (il peut alors le remplir n'importe comment), la valeur *off* désactive le pré-remplissage.

```
<label for="date_naissance">Date de naissance (au format JJ/MM/AAAA)
</label>
<input type="date" name="date_naissance" id="date_naissance"
autocomplete="bday">
```

9 Valider la soumission

Laisser l'utilisateur voir un récapitulatif des données renseignées pour qu'il s'assure qu'elles soient correctes et lui permettre de les corriger, avant qu'il soumette le formulaire.

L'utilisateur doit pouvoir confirmer ses actions et si possible les annuler.

10 Donner un retour à l'utilisateur

Il n'y a rien de plus frustrant de faire une action, comme cliquer sur un bouton, et de ne rien voir se passer.

Une fois que l'utilisateur a soumis le formulaire, lui indiquer que le formulaire a bien été soumis.

Toute action de l'utilisateur doit entraîner une réaction.

Dans le cas d'une validation serveur (non dynamique), le formulaire s'il contient des erreurs doit être ré-affiché à l'utilisateur **avec les données qu'il a renseignées** ainsi que les erreurs qui ont été commises.

Tabulation, ordre de tabulation et focus

1 Règles relatives à la tabulation et au focus

On peut utiliser la touche *Tab* pour naviguer dans la page. L'ordre dans lequel les éléments prendront le focus est généralement l'ordre dans lequel le contenu est structuré.

Quelques règles :

- lorsqu'un sous-menu est affiché au focus, **on doit pouvoir continuer à tabuler dans le sous-menu**
- au chargement d'une page, **le focus doit être placé en haut de la page**
- dans les applications réalisées avec des frameworks JS (Angular, React etc.), penser à bien modifier le titre de la page et à repositionner le focus en haut de page lorsque l'utilisateur est routé vers une autre page
- dans le cas d'une liste d'items dynamiques (un bouton permet d'ajouter un élément) ou d'un bouton « Voir plus » qui affiche des éléments supplémentaires (pagination), **le focus doit être repositionné sur le premier élément ajouté**



Ordre de tabulation du site Apple

2 L'attribut *tabindex*

L'attribut *tabindex* sur les **éléments interactifs seulement** peut prendre 2 valeurs :

- la valeur **-1** : l'élément interactif **ne peut pas être accédé à la navigation au clavier** (mais il peut prendre le focus programmatiquement)
- la valeur **0** : l'élément interactif **peut être accédé à la navigation au clavier et peut prendre le focus**

3 Les pièges au clavier

Un utilisateur qui navigue au clavier et qui active un composant comme une fenêtre modale par exemple, **la navigation au clavier reste bloquée au sein du composant et il ne peut pas en sortir en utilisant seulement son clavier.**

Il faut toujours prévoir un moyen de sortir du piège au clavier.

Dans le cas d'une fenêtre modale par exemple, l'utilisateur doit pouvoir la fermer en appuyant sur la touche *Échap*.

1 Les médias

Deux types de médias :

- les médias temporels : une vidéo, de la musique, un podcast etc.
- les médias non temporels : animation avec la balise `<canvas>` par exemple

Chaque média doit pouvoir être identifiable, un titre qui précède le média doit être présent.

3 Sous-titrage et transcription textuelle

Les sous-titres d'une vidéo doivent être synchronisés avec l'audio et reprendre le contenu sonore.

Une transcription textuelle reproduit à l'écrit le contenu sonore, les actions, du texte qui serait visible dans la vidéo, nécessaires à la compréhension du média.

Elle doit être accessible en-dessous du média ou accessible grâce à un lien ou un bouton.

2 Audio-description

Piste sonore d'une vidéo qui décrit ce qui se joue à l'écran : les actions importantes des protagonistes, du texte que serait en train de lire un protagoniste important pour la compréhension de la vidéo.

5 La Langue des Signes Française

Notamment, dans le cadre de certains programmes diffusés à la télévision, on peut voir une personne **interpréter (ou signer)** ce qui est dit en langue des signes française (LSF), pour que les personnes sourdes ou malentendantes puissent comprendre ce qui est dit.

La LSF est une **langue à part entière** au même titre que le français.

6 La vélotypie

Les vélotypistes utilisent un clavier spécifique, le clavier vélotype pour transcrire très rapidement ce qui est dit à l'oral.

La vélotypie est généralement utilisée pour produire des sous-titres pour les programmes en direct mais au contraire des sous-titres, ils ne reprennent pas mot pour mot le contenu sonore.

Elle a été utilisée durant les conférences de presse pendant l'épidémie de Covid-19 et les allocutions présidentielles mais sert également lors des audiences dans les tribunaux.



Clavier vélotype

7 Signer la musique

La musique peut être signée de multiples façons. Un interprète en LSF peut interpréter le contenu oral (mais les émotions et le sous-texte peuvent être perdues, le signage n'est pas une reproduction en LSF du contenu sonore à l'identique) mais aussi le rythme et la puissance du son.

C'est que fait Amber Galloway Gallego, une interprète en langues des signes américaine (ASL) qui a signé pour de grands artistes, comme Eminem.

8 Sons et vidéos déclenchés automatiquement

Ne pas déclencher de vidéo ou de son sans action de l'utilisateur.

Lui permettre de mettre en lecture ou de mettre en pause le média et il faut également qu'il puisse contrôler le volume sonore.

ARIA : rôles, attributs et motifs de conception

1 ARIA

ARIA (Accessible Rich Internet Applications) est un ensemble d'attributs qui permettent d'améliorer l'accessibilité des éléments et de créer des composants interactifs complexes accessibles.

Utiliser ARIA que **lorsque c'est nécessaire** :

- lorsque vous créez des composants interactifs
- pour définir des noms et descriptions accessibles
- lorsqu'un élément HTML n'est pas accessible (ce qui est rare)

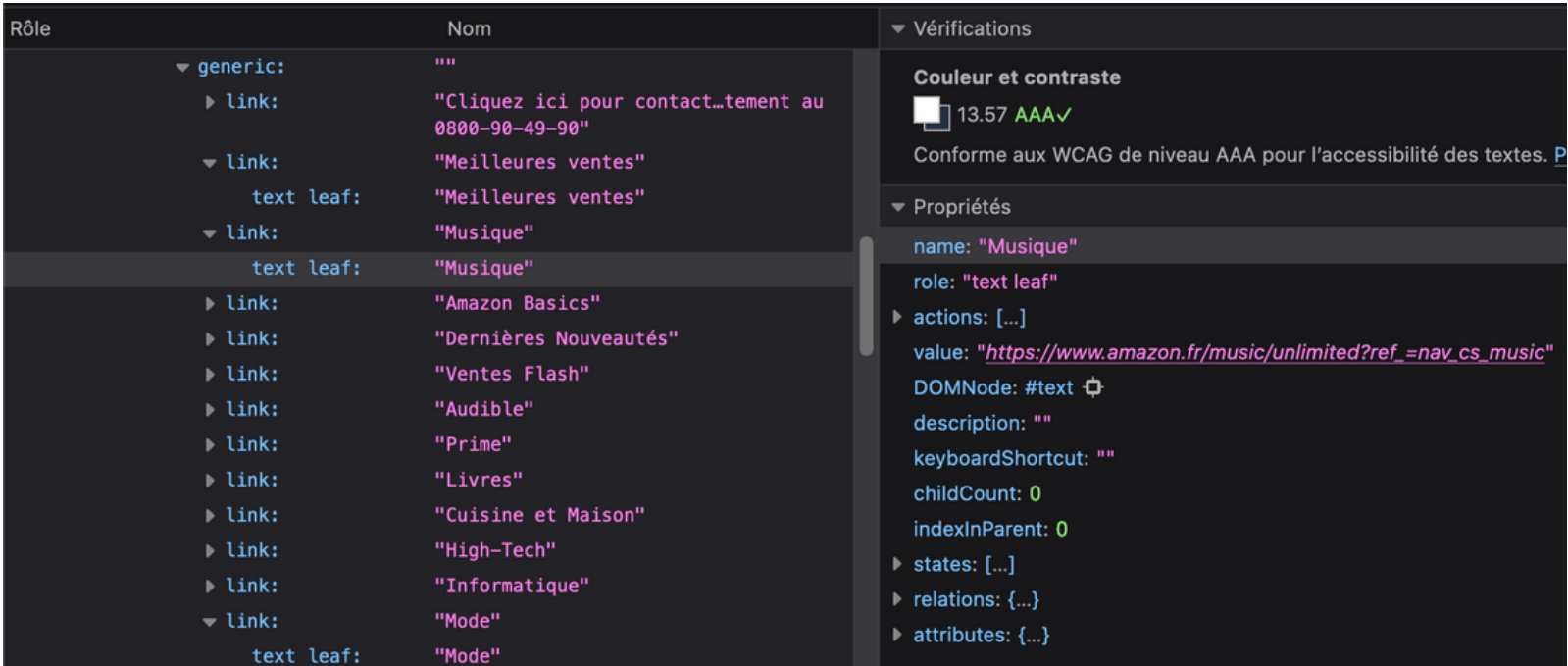
La page [ARIA states and properties](#) du site Mozilla Developer Network (MDN) liste les attributs et états ARIA possibles.

3 Les motifs de conception

La page [Patterns](#) du ARIA Authoring Practices Guide (APG) contient une liste de motifs de conception qui expliquent comment réaliser des composants interactifs accessibles en détaillant les interactions au clavier nécessaires ainsi que les rôles, attributs et états ARIA à utiliser.

2 L'arbre d'accessibilité

Comme pour le DOM (Document Object Model) qui est une représentation d'un document HTML sous la forme d'un arbre, l'arbre d'accessibilité représente les différents éléments HTML de façon simplifiée : leurs noms et descriptions accessibles, leurs rôles et leurs états.



Arbre d'accessibilité du site Amazon sur Firefox

ARIA : rôles, attributs et motifs de conception

3 L'attribut *role*

L'attribut *role* permet de redéfinir le rôle d'un élément : ainsi un `<span>` avec un attribut `role="button"` sera considéré comme un bouton **sans les comportements** d'un vrai bouton.

Changer le rôle ARIA d'un élément **ne le transforme pas**.

Sauf dans des cas très spécifiques, **il ne faut pas changer le rôle ARIA d'un élément** et utiliser systématiquement les éléments natifs HTML qui possèdent déjà les rôles et attributs ARIA nécessaires.

5 Les régions actives

Une région active est une zone de la page dont le contenu peut changer.

Des attributs ARIA spécifiques permettent de définir le comportement des technologies d'assistance quand le contenu de ces zones change.

4 Attributs de nom et de description accessible

- *aria-label* : définit le nom accessible d'un élément
- *aria-labelledby* : définit le nom accessible d'un élément grâce à d'autres éléments (liste d'identifiants d'éléments)
- *aria-describedby* : définit la description accessible d'un élément grâce à d'autres éléments (liste d'identifiants d'éléments)

6 L'attribut *aria-live*

L'attribut *aria-live* permet de signaler aux technologies d'assistance que le contenu d'une partie de la page va changer.

Trois valeurs possibles :

- *polite* : la technologie d'assistance attend avant de signaler à l'utilisateur la mise à jour du contenu
- *assertive* : la mise à jour du contenu est signalée immédiatement à l'utilisateur
- *off* : la contenu de la zone ne va pas changer (par défaut)

7 L'attribut *aria-atomic*

L'attribut *aria-atomic* indique aux technologies d'assistance si elles doivent décrire à l'utilisateur tout le contenu ou une partie du contenu quand il est modifié.

Trois valeurs possibles :

- *false* : la technologie d'assistance signale à l'utilisateur seulement le contenu mis à jour (par défaut)
- *true* : tout le contenu est décrit à l'utilisateur dès qu'il est mis à jour

Dans le cadre d'un outil de messagerie instantanée, si *aria-atomic* est à *false*, seul les nouveaux messages seront signalés à l'utilisateur.

8 L'attribut *aria-relevant*

L'attribut *aria-relevant* indique aux technologies d'assistance si quels sont les changements qu'elles doivent signaler à l'utilisateur.

Trois valeurs possibles :

- *additions* : la technologie d'assistance signale à l'utilisateur seulement le contenu ajouté (des éléments HTML ou du texte)
- *removals* : la technologie d'assistance signale à l'utilisateur seulement le contenu supprimé (des éléments HTML ou du texte)
- *text* : la technologie d'assistance signale à l'utilisateur seulement le texte ajouté
- *all* : ensemble des trois

9 L'attribut *aria-busy*

L'attribut *aria-busy* indique aux technologies d'assistance que le contenu est en train d'être mis à jour et que l'utilisateur doit être pas en être informé.

Deux valeurs possibles :

- *false* : la technologie d'assistance peut signaler à l'utilisateur le contenu
- *true* : la technologie d'assistance ne peut pas signaler à l'utilisateur le contenu (la TA attendra que l'attribut passe à *false*)

10 D'autres attributs ARIA utiles

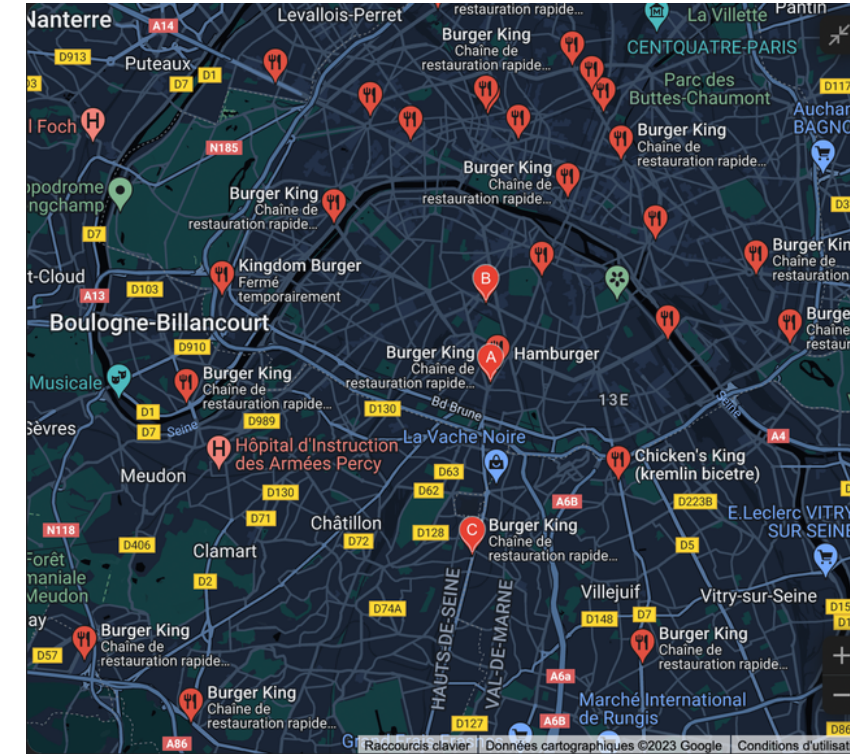
- *aria-controls* : liste d'identifiants d'éléments contrôlés par l'élément possédant l'attribut (par exemple un menu s'affichant au clic sur un bouton, le bouton contrôle le menu)
- *aria-current* : dans un conteneur d'éléments, définit l'élément actuel/sélectionné
- *aria-expanded* : indique que les éléments contrôlés (*aria-controls*) sont affichés/pliés ou cachés/dépliés (par exemple un menu, un accordéon)
- *aria-hidden* : ordonne aux technologies d'assistance d'ignorer un élément

Alternatives à un contenu inaccessible

1 Fournir une alternative accessible

Certains contenus, une carte Google Maps indiquant tous les restaurants Burger King par exemple, doivent être doublés par une alternative.

En l'occurrence, une liste reprenant tous les restaurants indiqués sur la carte.



Documents bureautiques

1 Documents bureautiques

Les documents bureautiques doivent également être accessibles : taille de police du texte et contrastes suffisants, alternatives aux images porteuses d'information etc.

Des outils permettent de vérifier l'accessibilité des documents, notamment des PDF, comme [PDF Accessibility Checker](#).

Toujours indiquer à l'utilisateur le format et la taille du document qu'il va télécharger. Cela bénéficie à tout le monde, rien de pire que de se retrouver avec un fichier de plusieurs centaines de méga-octets, qu'on pensait plus petit !

1 Les changements de contexte

Les changements de contexte est un changement dans la page qui peut être ignoré ou non compris par un utilisateur qui est incapable de voir la page dans sa globalité : une personne malvoyante utilisant une loupe pour mieux voir le contenu ou une personne utilisant un lecteur d'écran.

Il existe 4 types de changements de contexte :

- changement d'agent utilisateur : lien de téléchargement, ouverture d'un lecteur de PDF, d'un client e-mail par exemple
- changement d'espace de restitution : affichage d'une nouvelle page
- changement de focus : focus déplacé d'un endroit à un autre
- changement de contenu : du contenu qui change dynamiquement (des résultats de recherche par exemple)

Les changements de contexte doivent être notifiés à l'utilisateur (un texte explicatif) ou doivent être la conséquence d'une action qu'il a décidée (l'activation d'un lien ou d'un bouton).

Ressources

The A11Y Project

<https://www.a11yproject.com/>

W3C Web Accessibility Initiative (WAI)

<https://www.w3.org/WAI/>

Référentiel Général d'Amélioration de l'Accessibilité (RGAA)

<https://accessibilite.numerique.gouv.fr/>

web.dev

<https://web.dev/learn/accessibility>